

**(PART A – 2 Marks)**

**1. What does makemigrations and migrate commands do in Django?**

- **makemigrations** command
  - o Django detects changes in your models (models.py).
  - o Generates a new migration file inside the migrations folder.
  - o The file describes the changes (e.g., adding a new table or modifying a field).
- **migrate** command
  - o Applies the migrations (schema changes) to the actual database.
  - o Executes SQL commands to modify the database structure.
  - o Ensures that the database schema matches the Django models.

**2. Mention the files responsible for Model-View-Template pattern in Django Web Framework.**

- urls.py
- views.py
- models.py
- template.html

**3. What is the reason to choose popular web frameworks for web application development stack?**

- Streamlined development
- Speed and Efficiency
- Integrated Security
- Enhanced performance & scalability

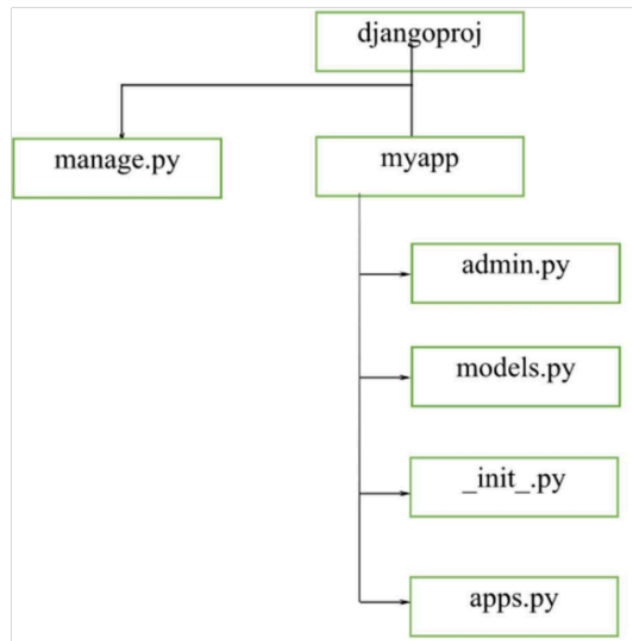
**4. List the various features of SQLite.**

- Serverless
- Light-weight
- Cross-platform
- Zero Configuration

**5. How does Django ensure that the submitted CSRF token is valid?**

- When the form is submitted, the token is sent along with the POST request.
- Django's CSRF middleware checks if the token matches the one stored in the user's session.
- If they don't match, the request is blocked with a 403 Forbidden error.

6. Draw the typical project structure of a Django application developed for managing a student database.



**(PART B – 13 Marks)**

**1. Draw the steps involved in developing a web application and explain each.**

Web application development is the process of using client-side and server-side programming to develop an application accessible over a web browser. It involves front-end (client-side) and back-end (server-side) development to create functional, user-friendly applications.



**Step 1 – Find a Genuine App Idea**

First, you start with an idea for a web app that solves a problem. Conduct brainstorming sessions and write down multiple ideas before deciding. Identify a gap in the market or an unmet need. Ensure your app idea provides value to users.

**Step 2 – Market Research**

Once you have an idea, it's time to understand the users. Both the business perspective and technical direction of the app depend on the people intended to serve. Use Google Trends, social media polls, and surveys to gather insights.

### **Step 3 – Define Functionality**

One core functionality that your app does best and which is the reason people should use your app must be isolated. Use a Feature Prioritization Matrix to rank features based on importance.

### **Step 4 – Sketch Your Web App Design**

A simple sketch is your first step to web application design. Just be sure to know where the buttons, text and images must be placed. This is also the stage to map out workflow. Use paper & pen or digital tools like Figma, Balsamiq, or Adobe XD.

### **Step 5 – Wireframes and Prototypes**

A wireframe tells how the app pages will look. The next step would be creating a web app prototype. When some interactivity is added to wireframes with limited functionality, that's called a prototype. Use tools like Figma, Adobe XD, or InVision for digital wireframes.

### **Step 6 – Time to Start Validating**

Round up a group of potential users. Get them to try out the web app prototype and begin gathering feedback. This feedback will help you immensely improve the app and make it more usable. Use UsabilityHub or Google Forms to collect feedback.

### **Step 7 – Choose Your Technology**

So far, we were dealing with the soft skills of web app development. Once you enter the development phase though, there are larger technical decisions to be made. Choose a scalable tech stack that suits your app's needs.

- MERN Stack for a JavaScript-based full-stack app.
- Django + PostgreSQL for a secure, data-heavy application.

### **Step 8 – Host Your Application on the Web**

You will need to choose a server to host your web based application. Buy a domain and set up an SSL certificate. Then, choose a cloud provider like Amazon (AWS) or MS Azure. You could also choose the Google Cloud Platform.

### **Step 9 – Deploy Your App**

Once you have the code and the host, you are ready to go live. Just deploy your app on to your hosting solution and your app is ready for your users. Launch the app, monitor performance and ensure the app is bug-free. Use Google Analytics to track user engagement.

**2. List all the steps involved in cloning a repository in Github, adding the files changed, commit, setting up remote-url and pushing the contents from Visual Studio Code to the repository back.**

Step-1: Activate environment

PS. C:/User/Admin> env1/scripts/activate

Step-2: Clone the repository in GitHub

(env1) C:/User/Admin> git clone https://github.com/sec/repo.git

Step-3: Change directory to the name of the repository cloned.

(env1) C:/User/Admin> cd repo

Step-4: Create a Django Project

(env1) C:/User/Admin/repo> django-admin startproject sam

Step-5: Change directory to the sub-folder

(env1) C:/User/Admin/repo> cd sam

Step-6: Create a new app

(env1) C:/User/Admin/repo/sam> python manage.py startapp myapp

Step-7: Click on settings.py and insert “import os”. Then insert ‘myapp’ in the configuration `INSTALLED_APPS`

Step-8: Check whether Django server is installed successfully

(env1) C:/User/Admin/repo/sam> python manage.py runserver 8000

Go to the URL ‘http://localhost:8000/’

Step-9: Create a migration file that contains code for the database schema of a model

(env1) C:/User/Admin/repo/sam> python manage.py makemigrations

Step-10: Apply all the migrations to the database

```
(env1) C:/User/Admin/repo/sam> python manage.py migrate
```

Step-11: Create admin

```
(env1) C:/User/Admin/repo/sam> python manage.py createsuperuser
```

Username: admin

Email address: admin@gmail.com

Password:

Password (again):

Superuser created successfully.

Step-12: Run Django admin

```
(env1) C:/User/Admin/repo/sam> python manage.py runserver 8000
```

Step-13: Copy the program in README.md and add the screenshot of output

Step-14: Inform Git that you want to include updates to a particular file in the next commit.

```
(env1) C:/User/Admin/repo/sam> git add -A
```

Step-15: Configure GitHub username

```
(env1) C:/User/Admin/repo/sam> git config user.name "USERNAME"
```

Step-16: Configure GitHub email

```
(env1) C:/User/Admin/repo/sam> git config user.email "EMAIL"
```

Step-17: Commit all the updates

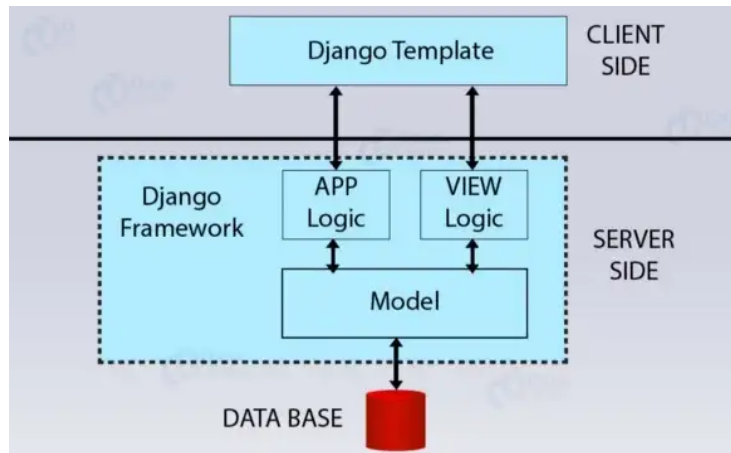
```
(env1) C:/User/Admin/repo/sam> git commit -m "Success"
```

Step-18: Push the changes in the branch to the Github repository

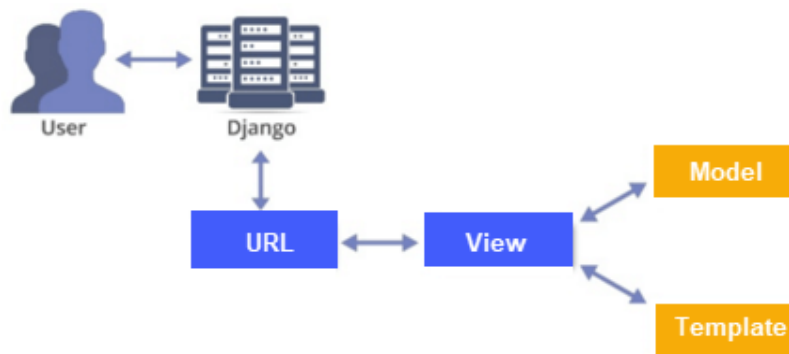
```
(env1) C:/User/Admin/repo/sam> git push https://github.com/USERNAME/repo.git
```

### 3. Give a detailed explanation of how a Django project is organized.

Django is a Python Web Application Framework. It is an open source framework for developing web applications in python. It is based on the Model-View-Template architecture and provides a separation of the data model from the business model and the user interface. Django can work with different databases without changing any code.



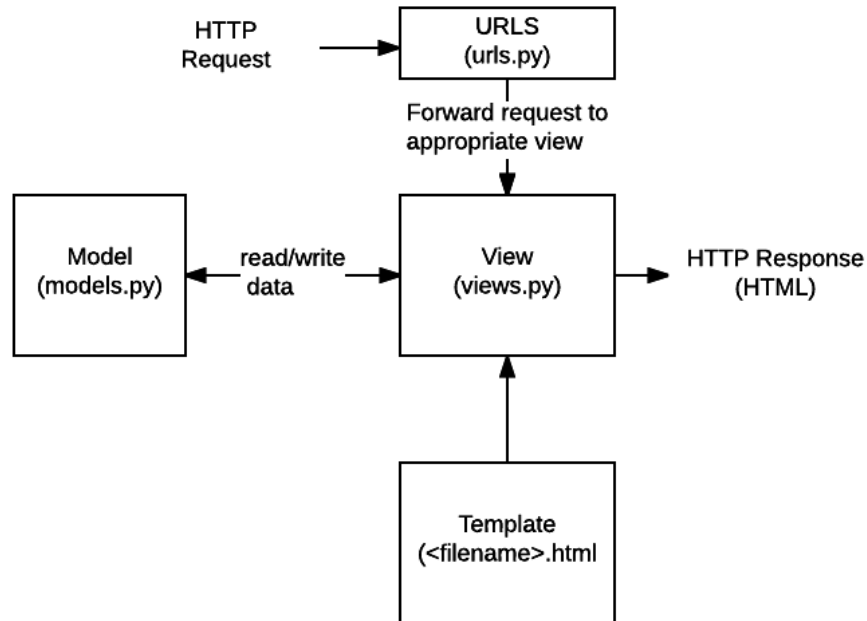
#### Django MVT Pattern



The diagram shows how all the components of the MVT pattern interact with each other to serve specific to a user request. A user requests a resource from Django. Django acts as a controller and checks the available resource in the URL. If URL maps, a view is called which interacts with the model and template. Django then responds to the user and sends a template as a response.

Model-View-Template architecture, otherwise known as MVT, is a software design pattern within Django that utilizes:

- Models to handle data logic and structure of your database
- Views to handle the applications logic and functionality
- Templates to handle the layout and structure of the user facing application



### (i) URLs.py

While it is possible to process requests from every single URL via a single function, it is much more maintainable to write a separate view function to handle each resource. A URL mapper is used to redirect HTTP requests to the appropriate view based on the request URL. The URL mapper can also match particular patterns of strings or digits that appear in a URL and pass these to a view function as data.

### (ii) view.py

A view is a request handler function, which receives HTTP requests and returns HTTP responses. Views access the data needed to satisfy requests via models, and delegate the formatting of the response to templates.

### (iii) Models.py

Models are Python objects that define the structure of an application's data, and provide mechanisms to manage (add, modify, delete) and query records in the database.

### (iv) Template

A template is a text file defining the structure or layout of a file (such as an HTML page), with placeholders used to represent actual content.

#### 4. Discuss any three web application frameworks.

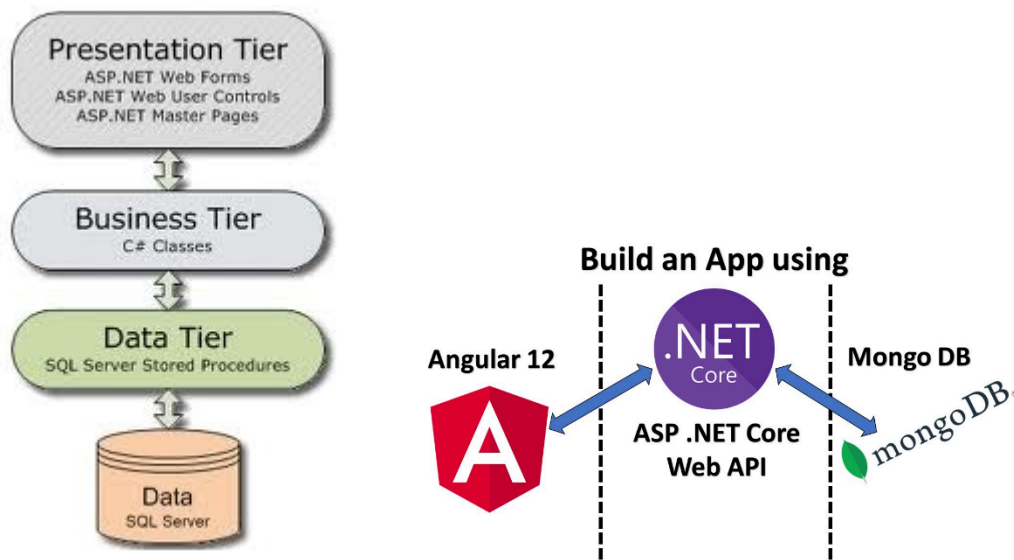
A web application framework is a software platform used to build and manage web applications efficiently. It provides tools, libraries, and best practices for faster development, scalability, and security.

##### (i).NET core

ASP.NET core is an open-source, server-side web-application framework designed for web development to produce dynamic web pages. It was developed by Microsoft to allow programmers to build dynamic web sites, applications and services. The name stands for Active Server Pages Network Enabled Technologies.

##### Features

- Supports C#
- Cross-platform (Windows, Linux, macOS)
- High-performance with ASP.NET Core
- Built-in dependency injection and middleware
- Supports Microservices & Cloud Computing (Azure)

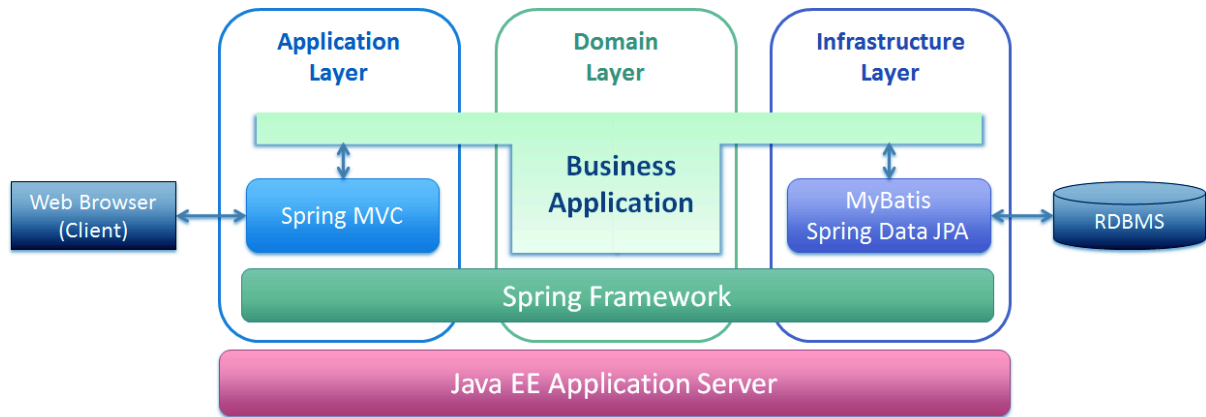


##### (ii) Spring Framework

Spring Framework is an application framework and inversion of control container for the Java platform. The framework's core features can be used by any Java application. The Spring Framework provides a comprehensive programming and configuration model for modern Java-based enterprise applications - on any kind of deployment platform.

##### Features

- Fast development with Spring Boot Auto-Configuration
- Built-in dependency management (Maven, Gradle)
- Supports Microservices architecture
- Integrated with Spring Security
- Works with Hibernate (ORM) and JPA for database handling



### (iii) MERN Stack

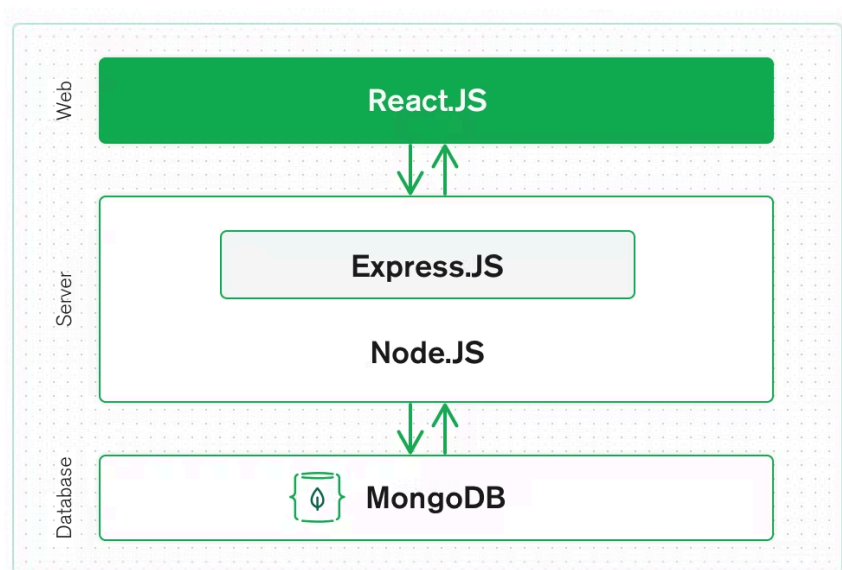
MERN Stack is a Javascript Stack that is used for easier and faster deployment of full-stack web applications. MERN Stack comprises of 4 technologies namely:

- MongoDB, Express, React and Node.js.

It is designed to make the development process smoother and easier. The MERN architecture allows you to easily construct a three-tier architecture (front end, back end, database) entirely using JavaScript and JSON.

#### Features

- Uses JavaScript for both frontend & backend
- MongoDB – NoSQL database for flexible data storage
- Express.js – Backend framework for API development
- React.js – Frontend UI framework
- Node.js – Server-side runtime for handling requests



**5. Develop a Django application that accepts height and weight as inputs, calculates the Body Mass Index (BMI) on the server side, and displays the result on both the webpage and the server console output.**

**Step-1: Create a Django Project & App**

- `django-admin startproject bmi_calculator`
- `cd bmi_calculator`
- `django-admin startapp bmiapp`

**Step-2: Configure settings.py**

- Open `bmi_calculator/settings.py` and add 'bmiapp' to `INSTALLED_APPS`.

**Step-3: Create a URL for the Web Form**

Open `bmi_calculator/urls.py` and modify.

```
from django.contrib import admin
from django.urls import path
from bmiapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.calculate_bmi, name='calculate_bmi'),
]
```

**Step-4: Define the View (Server Logic)**

- Open `bmiapp/views.py` and modify it.

```
from django.shortcuts import render

def calculate_bmi(request):
    if request.method == "POST":
        height = float(request.POST.get("height"))
        weight = float(request.POST.get("weight"))
        bmi = weight / ((height / 100) ** 2) # height in cm converted to m
        # Print BMI on server console
        print(f"Height: {height} cm, Weight: {weight} kg, BMI: {bmi:.2f}")
        return render(request, 'bmiapp/template.html', {'BMI': bmi})
```

### Step-5: Create a template.

- Create the file powerapp/templates/powerapp/template.html and enter the program.

```
<html>
<body>
  <h2>Body Mass Index (BMI) Calculator</h2>
  <form method="post">
    {% csrf_token %}
    <label>Height (cm):</label>
    <input type="text" name="height" required>

    <label>Weight (kg):</label>
    <input type="text" name="weight" required>

    <button type="submit">Calculate BMI</button>
  </form>
  <h3>Your BMI: {{ bmi }}</h3>
</body>
</html>
```

### Step-6: Run the Django server

- python manage.py runserver 8000

### Step-7: Output

- Open your browser and go to:  
<http://127.0.0.1:8000/>

### How This Works

- Gets weight and height from the user input using request.POST.get( ).
- Performs the calculation.
- Prints the output to the console for debugging.
- Passes the bmi value to the template to display it on the webpage.

### Expected Output

- Web Page:

Height (cm): 180  
Weight (kg): 81  
Your BMI: 25.0

- Server Console Output:

Height: 180.0 cm, Weight: 81.0 kg, BMI: 25

**6a. Create a model for a car inventory database that records information for 20 cars. Ensure the model includes at least seven relevant fields to describe each car.**

**models.py**

```
from django.db import models

from django.contrib import admin

class Car_Inventory(models.Model):

    Plate_No = models.CharField(max_length=20, primary_key=True)

    Car_Model = models.CharField(max_length=100)

    Car_Type = models.CharField(max_length=20)

    Mileage = models.IntegerField( )

    Engine_Type = models.CharField(max_length=15)

    Make_Date = models.DateField( )

    Car_Pic = models.ImageField( )

class Car_InventoryAdmin(admin.ModelAdmin):

    list_display = ('Plate_No', 'Car_Model', 'Car_Type', 'Mileage', 'Engine_Type',
                    'Make_Type', 'Car_Pic')
```

**admin.py**

```
from django.contrib import admin

from .models import Car_Inventory, Car_InventoryAdmin

admin.site.register(Car_Inventory, Car_InventoryAdmin)
```

**6b. Describe the various field types provided by Django models with examples.**

**A. CharField**

This field is used to store strings of up to a specified length. For example, a CharField might be used to store the name of a person, with a maximum length of 50 characters.

**B. IntegerField**

This field is used to store integers. For example, an IntegerField might be used to store the age of a person.

**C. FloatField**

This field is used to store floating-point numbers. For example, a FloatField might be used to store the height of a person.

**D. BooleanField**

This field is used to store boolean values. For example, a BooleanField might be used to indicate whether a user has agreed to the terms and conditions of a website.

**E. DateField**

This field is used to store dates. For example, a DateField might be used to store the date of birth of a person.

**F. DateTimeField**

This field is used to store dates and times. For example, a DateTimeField might be used to store the date and time a blog post was published.

**G. TimeField**

This field is used to store times. For example, a TimeField might be used to store the time that a movie starts.

**H. TextField**

This field is used to store large text blocks. For example, storing description of event.

**I. EmailField**

This field is used to store email addresses. For example, abc@xyz.com

**J. URLField**

This field is used to store URLs. For example, http://www.abc.com

**K. FileField**

This field is used to upload files to the server.

**L. ImageField**

This field is used to upload image files to the server. For example, an ImageField might be used to store the profile picture of a user.

**PART C – 15 Marks**

**1. Construct a Django model for Car\_Inventory with five different columns in which 'Car\_ID' field should be the primary key and prepare the required python models to insert unique rows into the table. Write the steps involved to set up the same in the admin interface.**

Step-1: Activate environment

PS. C:/User/Admin> env1/scripts/activate

Step-2: Clone the repository in GitHub

(env1) C:/User/Admin> git clone https://github.com/AAA/repo.git

Step-3: Change directory to the name of the repository cloned.

(env1) C:/User/Admin> cd repo

Step-4: Create a Django Project

(env1) C:/User/Admin/repo> django-admin startproject sam

Step-5: Change directory to the sub-folder

(env1) C:/User/Admin/repo> cd sam

Step-6: Create a new app

(env1) C:/User/Admin/repo/sam> python manage.py startapp myapp

Step-7: Edit settings.py for allowing all hosts and include 'myapp' in 'INSTALLED\_APPS'

Step-8: Check whether Django server is installed successfully

(env1) C:/User/Admin/repo/sam> python manage.py runserver 8000

Go to the URL 'http://localhost:8000/' and check.

Step-9: Open models.py and type the following

```
from django.db import models

from django.contrib import admin

class Car_Inventory(models.Model):

    Plate_No = models.CharField(max_length=20, primary_key=True)

    Car_Model = models.CharField(max_length=100)

    Car_Type = models.CharField(max_length=20)

    Mileage = models.IntegerField( )

    Engine_Type = models.CharField(max_length=15)

    Make_Date = models.DateField( )

    Car_Pic = models.ImageField( )

class Car_InventoryAdmin(admin.ModelAdmin):

    list_display = ('Plate_No', 'Car_Model', 'Car_Type', 'Mileage', 'Engine_Type',
                    'Make_Type', 'Car_Pic')
```

Step-10: Open admin.py and type the following

```
from django.contrib import admin

from .models import Car_Inventory, Car_InventoryAdmin

admin.site.register(Car_Inventory, Car_InventoryAdmin)
```

Step-11: Create a migration file that contains code for the database schema of a model

```
(env1) C:/User/Admin/repo/sam> python manage.py makemigrations
```

Step-12: Apply all the migrations to the database

```
(env1) C:/User/Admin/repo/sam> python manage.py migrate
```

Step-13: Create admin

```
(env1) C:/User/Admin/repo/sam> python manage.py createsuperuser
```

Username: admin

Email address: admin@gmail.com

Password:

Password (again):

Superuser created successfully.

Step-14: Migrate database schema of Book model to 'myapp'

```
(env1) C:/User/Admin/repo/sam> python manage.py makemigrations myapp
```

Step-15: Apply all the migrations to the database in 'myapp'

```
(env1) C:/User/Admin/repo/sam> python manage.py migrate myapp
```

Step-16: Run Django admin

```
(env1) C:/User/Admin/repo/sam> python manage.py runserver 8000
```

Go to the URL 'http://localhost:8000/admin'

Step-17: After logging into the Admin account, enter the details.

## 2. Explain in detail about various options in modelAdmin.

The ModelAdmin is very flexible. It has several options for dealing with customizing the interface. All options are defined on the ModelAdmin subclass.

### A. ModelAdmin.actions

A list of actions to make available on the change list page.

### B ModelAdmin.actions\_on\_top

Controls where on the page the actions bar appears. By default, the admin changelist displays actions at the top of the page.

Example

```
actions_on_top = True;
```

### C. ModelAdmin.actions\_on\_bottom

It displays the admin changelist actions at the bottom of the page.

Example

```
actions_on_bottom = True;
```

### D. ModelAdmin.actions\_selection\_counter

Controls whether a selection counter is displayed next to the action dropdown. By default, the admin changelist will display it.

Example

```
actions_selection_counter = True;
```

### E. ModelAdmin.date\_hierarchy

Set date\_hierarchy to the name of a DateField or DateTimeField in your model, and the change list page will include a date-based drill down navigation by that field.

Example

```
date_hierarchy = "pub_date"
```

### F. ModelAdmin.exclude

This attribute, if given, should be a list of field names to exclude from the form. For example, let's consider the following model,

```
class Author(models.Model):  
    name = models.CharField(max_length=100)  
    title = models.CharField(max_length=3)  
    birth_date = models.DateField()
```

If you want a form for the Author model that includes only the name and title fields, you would specify fields or exclude like this:

```
class AuthorAdmin(admin.ModelAdmin):  
    fields = ["name", "title"]  
    (or)  
class AuthorAdmin(admin.ModelAdmin):  
    exclude = ["birth_date"]
```

### **G. ModelAdmin.list\_display**

Set `list_display` to control which fields are displayed on the change list page of the admin.

Example

```
list_display = ["first_name", "last_name", "regno"]
```

### **H. ModelAdmin.list\_display\_links**

Use `list_display_links` to control which fields in `list_display` should be linked to the “change” page for an object.

Example

```
list_display_links = ["regno"]
```

### **I. ModelAdmin.fieldsets**

Set `fieldsets` to control the layout of admin “add” and “change” pages. `fieldsets` is a list of two-tuples, in which each two-tuple represents a `<fieldset>` on the admin form page. The two-tuples are in the format `(name, field_options)`, where `name` is a string representing the title of the fieldset and `field_options` is a dictionary of information about the fieldset, including a list of fields to be displayed in it.

Example

```
class sampleAdmin(admin.ModelAdmin):  
    fieldsets = [  
        None,  
        {  
            "fields": ["regno", "name", "age"],  
        },  
        "Advanced options",  
        {  
            "fields": ["address", "mobile"],  
        },  
    ]
```